



Actividad 4

Threading

Entrega

- **Lugar:** Repositorio personal de GitHub — Carpeta: Actividades/AC4
- **Fecha máxima de entrega:** 1 de Octubre 17:10
- **Ejecución de actividad:** La Actividad será ejecutada **únicamente** desde la terminal del computador. Los *paths* relativos utilizados en la Actividad deben ser coherentes con esta instrucción, y no pueden modificarse.

Importante: Antes de comenzar, comprueba que Git este funcionando correctamente en tu repositorio privado. Para esto, **sube los archivos base de la actividad de inmediato** (*add*, *commit*, *push*). Se espera que en esta actividad (así como en las demás actividades y tareas) utilices Git a lo largo de **todo tu desarrollo** como una herramienta, no sólo como un método de entrega. Es por esto que recomendamos enfáticamente que vayas subiendo tus cambios constantemente (*push*), ya que **problemas de último minuto** relacionados con la entrega y Git **no serán considerados**.

Introducción

Estas en el restaurante más caótico de la ciudad. Hay una pila de pedidos esperando, 4 cocineros listos para trabajar, y un local con serios problemas de infraestructura.



Estas son las reglas del lugar:

- Cada cocinero tiene **su propia cola de pedidos** y los atiende **en orden de llegada**, de uno en uno: no empieza un pedido nuevo hasta terminar el que tiene entre manos.
- Nadie toca nada hasta que el local abre. Cuando se da la **señal de apertura**, los 4 cocineros empiezan a trabajar **al mismo tiempo**.
- Un plato se prepara en tres etapas, siempre **en este orden**: se pican los ingredientes, se cocina y se emplata. Un plato emplataado se considera **entregado**, y el momento exacto de la entrega importa: el restaurante quiere saber cuánto demora cada pedido desde que comienza a ser atendido hasta que es entregado.
- Picar y emplatar se hacen en el mesón de cada cocinero, así que **varios cocineros pueden picar o emplatar simultáneamente**.
- Cocinar es otra historia: en **todo el restaurante hay una sola estufa**. Mientras un cocinero la está usando, los demás no pueden cocinar, aunque ya tengan sus ingredientes picados y listos; tienen que esperar su turno.
- La cocción es delicada. A veces un plato **se quema** y hay que rehacerlo, lo que toma tiempo extra. El cocinero **no suelta la estufa** mientras rehace el plato: el resto sigue esperando.
- El servicio no dura para siempre. Pasada una cierta cantidad de segundos desde la apertura, **la cocina cierra**. Un cocinero que ya comenzó a preparar un pedido debe terminarlo y entregarlo, pero **nadie puede comenzar un pedido nuevo después del cierre**. Por esta razón, al finalizar el servicio, el restaurante necesita identificar cuáles pedidos quedaron pendientes en la cola de algún cocinero y que nunca llegaron a ser entregados.

Tu trabajo es simular este servicio usando **Threading**.

Archivos

El código base tiene 5 archivos. Preocúpate de subir al repositorio los archivos con el *tag* **Entregar**. El resto no es necesario que los entregues.

```
Actividades/  
├── AC4/  
│   ├── clases.py  
│   ├── main.py  
│   ├── parametros.py  
│   ├── eventos_random.py  
│   └── utilidades.py
```

Los archivos que deberás utilizar se describen a continuación.

- **Modificar** **Entregar** `clases.py`: Contiene las tres clases principales de la simulación. En este archivo deberás completar la lógica necesaria para que los cocineros puedan trabajar de manera concurrente y coordinarse durante el servicio. En particular, deberás completar la implementación de `Cocinero` y de los métodos `Cocina.cerrar_cocina` y `Cocina.simular_servicio`, además de definir los atributos o recursos compartidos que consideres necesarios para cumplir las reglas descritas en el enunciado.

Puedes agregar los métodos y atributos que consideres necesarios para implementar la simulación. Sin embargo, debes mantener los nombres, parámetros y valores de retorno de los métodos ya existentes, ya que estos son utilizados por el resto del código base.

- **Modificar** **Entregar** `main.py`: Crea la cocina, ejecuta la simulación y muestra el reporte final utilizando la información retornada por el método `simular_servicio`.

Este es el archivo desde el cual será ejecutada tu actividad. Debes completar lo necesario para iniciar la simulación (ver Parte 3).

- **No modificar** `parametros.py`: contiene las constantes utilizadas por la simulación, entre ellas `NUM_COCINEROS`, `PEDIDOS_POR_COCINERO`, `DURACION_SERVICIO`, `NOMBRES_COCINEROS` y `MENU`.
- **No modificar** `eventos_random.py`: contiene las funciones que determinan los tiempos y eventos aleatorios de la simulación.
- **No modificar** `utilidades.py`: contiene funciones auxiliares para registrar mensajes en consola, iniciar el cronómetro y generar el reporte final.

Modelo de datos

La simulación utiliza las clases `Pedido`, `Cocinero` y `Cocina`, definidas en el archivo `clases.py`.

El código base incluye atributos, métodos y *docstrings* que describen la responsabilidad de cada clase. Durante la actividad podrás agregar los atributos y métodos que consideres necesarios para implementar el comportamiento solicitado en las secciones siguientes.

A continuación se resumen los elementos que ya se encuentran definidos.

- **No modificar** `class Pedido`:

Representa un pedido realizado por un cliente. Cada pedido posee un identificador único, el nombre del plato solicitado y los instantes de inicio y entrega del pedido.

Sus atributos son:

<code>self.id_pedido</code>	<code>int</code> que identifica al pedido.
<code>self.plato</code>	<code>str</code> con el nombre del plato solicitado.
<code>self.inicio</code>	<code>datetime</code> correspondiente al instante en que comienza su preparación.
<code>self.entrega</code>	<code>datetime</code> correspondiente al instante en que termina su preparación.

Su método `__repr__` ya se encuentra implementado. Por ejemplo, un pedido puede mostrarse de la siguiente forma:

Pedido #3 (Hamburguesa doble)

■ **Modificar** `class Cocinero:`

Representa a uno de los cocineros del restaurante.

El código base define los siguientes atributos y métodos:

<code>self.nombre</code>	<code>str</code> con el nombre del cocinero.
<code>self.pedidos</code>	<code>deque</code> que contiene los pedidos que tiene asignados y que aún no han comenzado a ser atendidos.
<code>asignar_pedido(pedido)</code>	Método ya implementado que agrega un pedido al final de la cola del cocinero.

Deberás completar esta clase con el comportamiento necesario para que cada cocinero pueda realizar su turno según las reglas descritas en la actividad.

■ **Modificar** `class Cocina:`

Representa el servicio completo del restaurante y mantiene la información compartida por los distintos cocineros.

El código base define los siguientes atributos y métodos:

<code>self.duracion_servicio</code>	<code>float</code> que indica cuántos segundos permanece abierta la cocina.
<code>self.cocineros</code>	<code>dict</code> que relaciona el nombre de cada cocinero con su instancia de <code>Cocinero</code> .
<code>self.pedidos</code>	<code>list</code> que contiene todos los pedidos creados para el servicio.
<code>contratar_cocineros(cantidad)</code>	Método ya implementado que crea los cocineros del turno.
<code>recibir_pedidos(cantidad)</code>	Método ya implementado que genera los pedidos del servicio.
<code>repartir_pedidos()</code>	Método ya implementado que distribuye los pedidos entre los cocineros.
<code>cerrar_cocina()</code>	Método que deberás completar para representar el cierre de la cocina.
<code>simular_servicio()</code>	Método que deberás completar para coordinar el servicio completo del restaurante.

El método `__init__` ya crea los cocineros, genera los pedidos y los reparte. Puedes agregar los atributos que necesites para implementar la coordinación del servicio.

Las siguientes funciones se encuentran en el archivo `eventos_random.py`.

No modificar `eventos_random.py`

Este archivo contiene las funciones utilizadas para determinar los tiempos y eventos aleatorios del servicio. **Debes utilizar las cinco funciones** y estas constituyen la única fuente de aleatoriedad permitida para la simulación.

Función	Retorna	Significado
<code>random_tiempo_picado()</code>	<code>float</code> entre 1 y 2	Tiempo utilizado para picar
<code>random_tiempo_coccion()</code>	<code>float</code> entre 2 y 3	Tiempo utilizado para cocinar
<code>random_tiempo_emplatado()</code>	<code>float</code> entre 0.5 y 1	Tiempo utilizado para emplatar
<code>random_se_quema_plato()</code>	<code>bool</code>	Indica si el plato se quema o no
<code>random_tiempo_extra()</code>	<code>float</code> entre 1 y 2	Tiempo adicional por plato quemado

La función `random_se_quema_plato()` retorna `True` con una probabilidad de 25 %.

Parte 1. El turno de los cocineros

En esta parte deberás implementar en las acciones que realiza un cocinero durante su turno. Los cuatro cocineros deberán realizar estas acciones de forma concurrente y coordinarse para respetar las reglas del restaurante.

1.1 Inicio y atención de pedidos

Al comenzar su turno, cada cocinero debe esperar hasta que el restaurante abra antes de empezar a trabajar.

Una vez abierto el restaurante, cada cocinero atenderá los pedidos de `self.pedidos` en el mismo orden en que fueron asignados.

Cuando comienza un pedido, debe removerlo de la cola y registrar el tiempo en `pedido.inicio`. Desde ese momento, el pedido se considera **iniciado** y deberá terminarse incluso si la cocina cierra durante su preparación.

Antes de comenzar cada nuevo pedido, el cocinero debe comprobar si la cocina continúa abierta. Si ya cerró, deberá finalizar su turno sin comenzar nuevos pedidos.

1.2 Preparación de los pedidos

Cada pedido debe pasar, en este orden, por las siguientes etapas:

1. Picar los ingredientes.
2. Cocinar el plato.
3. Emplatar el plato.

Los tiempos de cada etapa son determinados por las funciones de `eventos_random.py`. Picar y emplatar son actividades independientes, por lo que distintos cocineros pueden realizarlas simultáneamente.

Para cocinar existe una única estufa en todo el restaurante. Por lo tanto, solo un cocinero puede estar cocinando en ella a la vez. Si la estufa está siendo utilizada, los demás cocineros que necesiten cocinar deberán esperar hasta que quede disponible.

Tu implementación debe garantizar que esta restricción se cumpla durante toda la simulación.

Durante la cocción, un plato puede quemarse según lo indicado por `eventos_random.py`. Si esto ocurre, el cocinero debe informar el evento en consola y esperar el tiempo adicional necesario para rehacerlo, sin dejar disponible la estufa durante este proceso.

1.3 Entrega y final del turno

Después de emplatar, el pedido se considera entregado. En ese momento, debes registrar el instante de entrega en `pedido.entrega`.

Todos los cocineros comparten este registro, por lo que tu implementación debe garantizar que las entregas se registren correctamente, incluso cuando distintos cocineros terminen pedidos de manera concurrente.

Un cocinero finaliza su turno cuando ya no tiene pedidos pendientes o cuando, después de terminar el pedido que estaba preparando, encuentra que la cocina ya está cerrada.

Los pedidos que permanezcan en `self.pedidos` al finalizar su turno corresponden a pedidos que nunca comenzaron a ser atendidos.

Antes de retirarse, el cocinero debe informar mediante la consola si atendió todos sus pedidos o cuántos quedaron pendientes.

Parte 2. Simulación del servicio

El método `simular_servicio` es responsable de coordinar el funcionamiento completo del restaurante: desde preparar a los cocineros para comenzar sus turnos hasta esperar que todos hayan terminado y construir el resultado final de la simulación.

2.1 Inicio del servicio

Antes de abrir el restaurante, debes iniciar la ejecución concurrente de todos los cocineros. Cada uno deberá quedar esperando hasta que el restaurante abra antes de comenzar a atender sus pedidos.

Una vez que todos los cocineros estén preparados, debes realizar, en este orden, las siguientes acciones:

1. Llamar a `iniciar_cronometro()`.
2. Informar mediante consola que el restaurante ha abierto.
3. Comunicar a los cocineros que pueden comenzar sus turnos.

Tu implementación debe asegurar que ningún cocinero comience a preparar un pedido antes de la apertura del restaurante.

2.2 Cierre de la cocina

Deberás completar el método `cerrar_cocina()` para representar el cierre del servicio. Cuando este método sea ejecutado, debe informar mediante la consola que la cocina ha cerrado y actualizar el estado necesario para que los cocineros sepan que no pueden comenzar nuevos pedidos.

La cocina debe cerrar automáticamente después de `self.duracion_servicio` segundos desde la apertura.

Durante ese tiempo, los cocineros deben poder continuar trabajando normalmente. Por lo tanto, la coordinación del cierre no debe impedir que el resto de la simulación continúe ejecutándose.

Una vez transcurrido el tiempo correspondiente, se debe ejecutar `self.cerrar_cocina()`.

2.3 Fin de los cocineros

Después de abrir el restaurante y programar su cierre, el *thread* principal debe esperar a que **todos los cocineros hayan terminado sus turnos**.

Para esto puedes utilizar el método correspondiente disponible en los objetos `Thread`.

No debes construir los resultados finales mientras todavía existan cocineros trabajando, ya que aún podrían existir pedidos en preparación o información de entrega pendiente de registrar.

2.4 Retorno

Una vez finalizados todos los turnos, `simular_servicio` debe retornar una tupla con exactamente dos elementos:

1. Un `dict[int, timedelta]` que indique, para cada pedido **entregado**, cuánto tiempo transcurrió desde que comenzó a ser atendido hasta su entrega.

Las llaves de este diccionario deben corresponder a los `id_pedido` de los pedidos entregados.

Para obtener cada `timedelta`, puedes restar el `datetime` almacenado en `pedido.inicio` al `datetime` almacenado en `pedido.entrega`.

2. Una `list[Pedido]` que contenga todos los pedidos que **no fueron entregados**.

Parte 3. Iniciar la simulación (main.py)

Tu simulación está lista, pero aún debes ponerla en marcha. En `main.py`, crea una instancia de `Cocina` utilizando la configuración definida en `parametros.py`, ejecuta su método `simular_servicio` y guarda el resultado retornado.

No es necesario crear manualmente los cocineros ni los pedidos: la instancia de `Cocina` se encarga de hacerlo utilizando los parámetros entregados.

Finalmente, utiliza el resultado de la simulación para generar el reporte final, siguiendo la estructura ya presente en `main.py`.

La consola

La consola es nuestra herramienta para saber qué ocurre en la simulación, por lo que sus mensajes también se usarán para corregir. A partir de una ejecución debe ser posible reconstruir el servicio completo e identificar:

- Cuándo abre y cierra el restaurante.
- Qué pedido está preparando cada cocinero y en qué etapa se encuentra.
- Cuándo cada cocinero toma y libera la estufa.
- Cuándo un plato se quema y debe rehacerse.
- Cuándo se entrega cada pedido.
- Cómo termina el turno de cada cocinero, indicando si quedan pedidos pendientes.

Para imprimir mensajes desde los distintos *threads* debes utilizar `log(emisor, mensaje)`, disponible en `utilidades.py`. Esta función evita que los mensajes de distintos *threads* se mezclen al imprimirse simultáneamente.

Los mensajes asociados a la estufa son especialmente importantes: entre el momento en que un cocinero indica que **entra a la estufa** y el momento en que la **libera**, ningún otro cocinero puede indicar que entra a ella. Si esto ocurre, significa que dos cocineros están utilizando la estufa al mismo tiempo y la simulación es incorrecta.

Notas

- Solo se permite importar la librería `threading`, `time` y los módulos entregados junto con la actividad. No se permite utilizar otras librerías externas.
- La actividad será evaluada utilizando únicamente las versiones modificadas de `main.py` y `clases.py`. Los cambios realizados en otros archivos no serán considerados durante la corrección.
- Recuerda que debes entregar la actividad en **tu repositorio personal de Git**, dentro de la rama (*branch*) por defecto `main`.
- Se recomienda desarrollar la actividad siguiendo el orden de las secciones del enunciado.

Objetivos de la actividad

- Aplicar conocimientos de *Threading* para ejecutar tareas de manera concurrente.
- Diseñar una solución concurrente a partir de las restricciones del problema, identificando qué elementos requieren coordinación y cuáles requieren acceso exclusivo.

Puntaje por sección

Los criterios indican los aspectos principales que serán evaluados. Pueden subdividirse durante la corrección.

Criterio	Puntaje
Parte 1: El turno de los cocineros	
Los cocineros realizan sus turnos de manera concurrente, de modo que puedan avanzar en sus tareas de forma independiente mientras corresponda.	0,5 pts
Los cocineros esperan correctamente la apertura del restaurante antes de comenzar a trabajar y respetan su cierre, terminando los pedidos ya iniciados sin comenzar nuevos pedidos después de este.	0,5 pts
Cada cocinero atiende sus pedidos en el orden correspondiente y realiza correctamente las etapas de picado, cocción y emplatado, utilizando los tiempos y eventos provistos.	0,5 pts
El uso de la estufa respeta que existe una sola disponible para todos los cocineros. Mientras está siendo utilizada, ningún otro cocinero puede cocinar en ella, incluyendo cuando un plato debe ser rehecho.	0,5 pts
Los instantes de inicio y entrega de cada pedido se registran correctamente, permitiendo calcular su tiempo de preparación. Además, cada cocinero informa correctamente cómo finaliza su turno.	0,5 pts
Parte 2: Simulación del servicio	
<code>Cocina.simular_servicio</code> coordina correctamente el inicio del servicio, asegurando que los cocineros estén preparados antes de comunicar la apertura del restaurante.	0,4 pts
La cocina cierra automáticamente después de <code>self.duracion_servicio</code> segundos, sin impedir que los cocineros continúen trabajando normalmente mientras transcurre el tiempo.	0,4 pts
La simulación espera correctamente hasta que todos los cocineros hayan finalizado sus turnos antes de construir los resultados del servicio.	0,4 pts
<code>simular_servicio</code> construye y retorna correctamente la información de los pedidos entregados, sus tiempos correspondientes y los pedidos que quedaron sin atender.	0,8 pts
Parte 3: Iniciar la simulación	
<code>main.py</code> crea correctamente la cocina, ejecuta el servicio y utiliza la información retornada para generar el reporte final.	0,5 pts
Descuentos: Se podrá aplicar un máximo de 1 punto de descuento por problemas de legibilidad, incumplimiento de PEP 8 o de las restricciones indicadas. Se aplicará el descuento según la siguiente escala: ▪ Hasta 3 errores: 0 puntos de descuento. ▪ Entre 4 y 7 errores: 0,5 puntos de descuento. ▪ 8 o más errores: 1 punto de descuento.	
Total	5,0 pts